

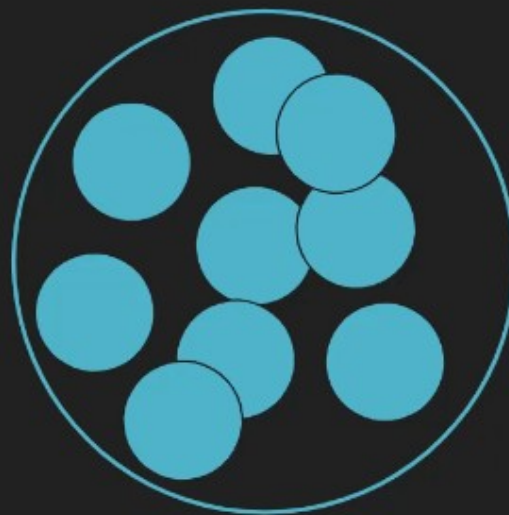
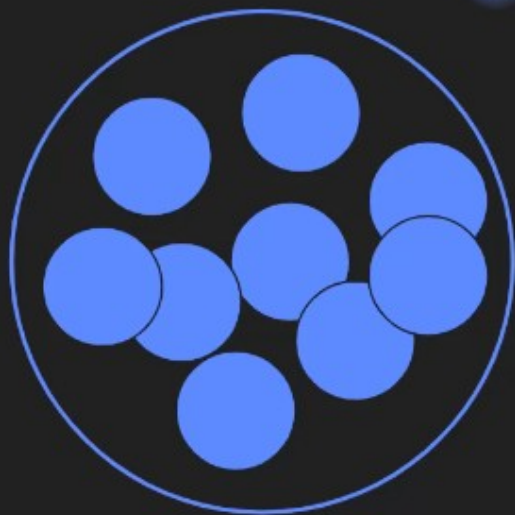
Purpose and interpretation of the t-test

In this video, you will learn

- The general idea of a t-test.
- The basic formula of the t-test.
- How to derive a p-value from a t-test.
- How to make your t-statistic large.

T-test: the main idea

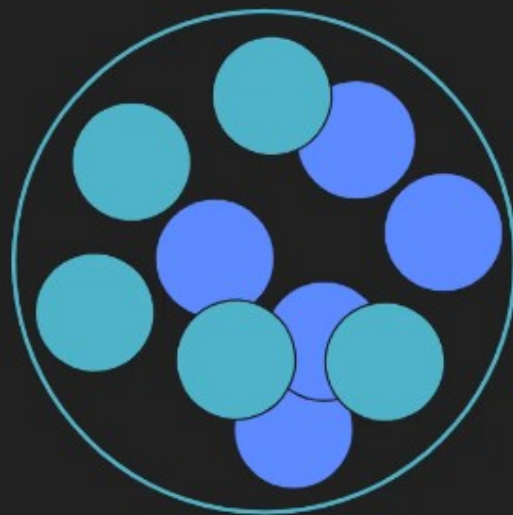
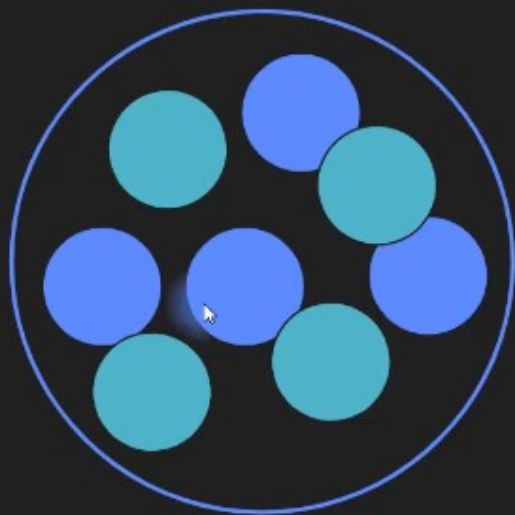
$H_A = \text{red} \text{ vs. } \text{blue}$



The purpose of the t-test is to compare data (typically means) between two groups.

T-test: the main idea

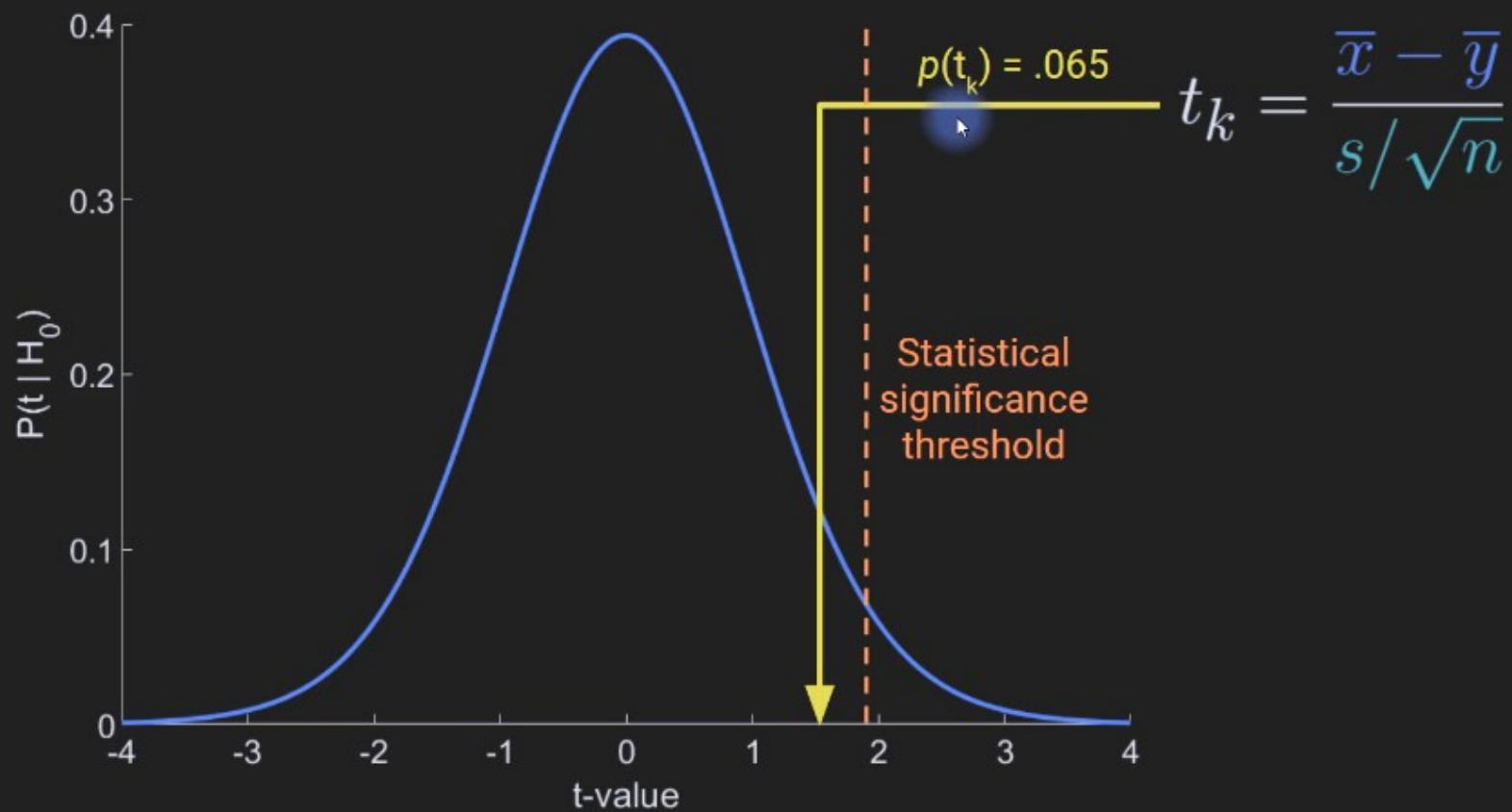
$$H_0 = \text{blue circle} == \text{red circle}$$



T-test: the formula

$$t_k = \frac{\bar{x} - \bar{y}}{s/\sqrt{n}} = \frac{\text{Difference of means}}{\text{Standard deviations}}$$

P-values from t-values



$$t_k = \frac{\bar{x} - \bar{y}}{s/\sqrt{n}} = \frac{(\bar{x} - \bar{y})\sqrt{n}}{s}$$

Increase the group differences

Reduce variances

Increase sample size

One-sample t-test

In this video, you will learn

- The purpose and assumptions behind one t-test.
- The formula of the one-sample t-test.

One-sample t-test: purpose

Test whether a set of numbers could have been drawn from a distribution with a specified mean.

This time we are not comparing two groups, we compare a group with a distribution.

One-sample t-test: example application

Test whether the IQ of a group of students is significantly different from 100.

(Formal statement: Estimate the probability that a certain group of students has an average IQ of 100.)

One-sample t-test: formula

$$t_{n-1} = \frac{\bar{x} - \mu}{s/\sqrt{n}}$$

$\bar{x}$ Sample mean

μ H_0 value

s Sample standard deviation

n Number of data points

$n-1$ Degrees of freedom

One-sample t-test: assumptions

1. Data are numeric (not categorical), ideally interval or ratio (discrete is probably OK).
2. Data are independent from each other.
3. Data are randomly drawn from the population to which generalization should be made.
4. Mean and standard deviation are valid descriptors of central tendency and dispersion (i.e., data are approximately normally distributed).

```

import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## generate the data
# parameters
N = 20 # sample size
popMu = .5 # true population mean
data = np.random.randn(N) + popMu

# let's see what the data look(s) like
plt.plot(data, 'ko-', markerfacecolor='w', markersize=10)
plt.xlabel('Data index')
plt.ylabel('Data value')
plt.show()

### question: Should there be lines in this plot?

## "manual" t-test

# the null-hypothesis value
H0val = 0

# compute the t-value
t_num = np.mean(data) - H0val
t_den = np.std(data) / np.sqrt(N)
tval = t_num / t_den

# degrees of freedom
df = N-1

# p-value
pval = 1-stats.t.cdf(abs(tval), df)

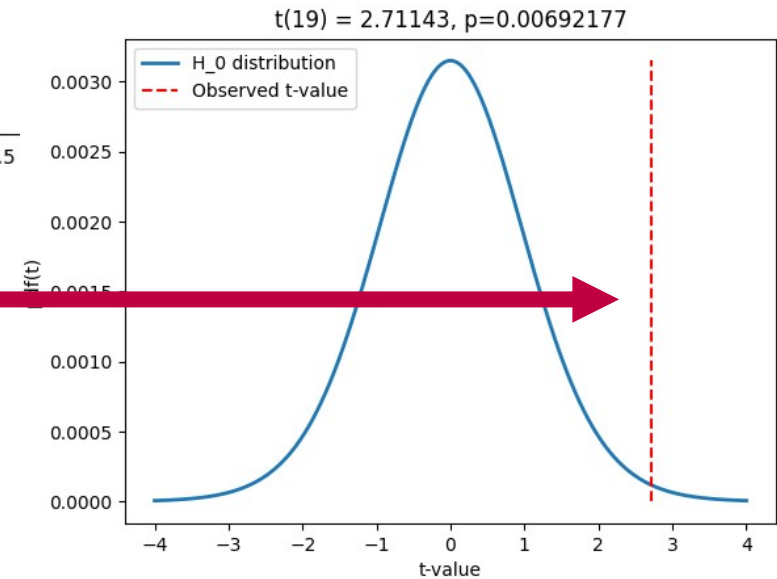
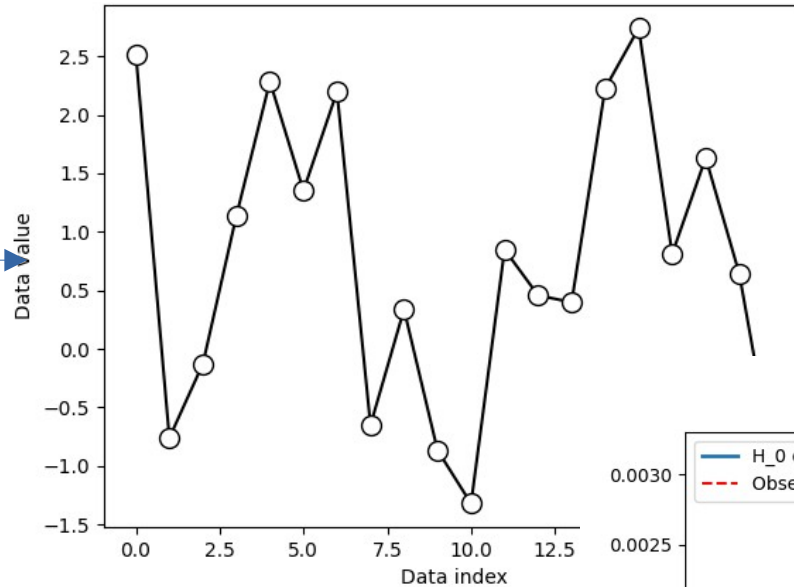
# show the H0 parameter distribution and observed t-value
x = np.linspace(-4, 4, 1001)
tdist = stats.t.pdf(x, df) * np.mean(np.diff(x))

plt.plot(x, tdist, linewidth=2)
plt.plot([tval, tval], [0, max(tdist)], 'r--')
plt.legend(('H_0 distribution', 'Observed t-value'))
plt.xlabel('t-value')
plt.ylabel('pdf(t)')
plt.title('t(%g) = %g, p=%g'%(df, tval, pval))
plt.show()

## now using the Python function
t, p = stats.ttest_1samp(data, H0val)

print(t, p)
# do these values match our manually computed values?

```



```

warnings.warn( 'Unable to import Axes3D. This may b
2.642773377835189 0.01604771669702498
atalet@uzgem:~/workspace/YZ_EGITIM/ozetler/MATH-02-M

```

Two-samples t-test

In this video, you will learn

- How the two-samples t-test is a simple extension of the one-sample t-test.
- That different denominators depend on the characteristics of the data groups.

Two-samples t-test: purpose

Test whether two sets of numbers could have been drawn from the same distribution.

Two-samples t-test: example application

Test whether self-reported stress levels changed after 6 weeks of “social distancing.”

(Formal statement: Estimate the probability that self-reported stress levels before and after 6 weeks of social distancing were drawn from the same distribution.)

Two-sample t-tests: formulas

There are several two-sample t-test formulas.

The numerator is always the same.

The denominator depends on whether the groups are paired or unpaired, have equal or unequal variance, and have matched or different sample sizes.

Explanations of t-test types

Paired or unpaired: Whether the two groups of data are drawn from the same or different individuals.

Paired: The same individuals self-report their stress levels before vs. after social distancing.

Unpaired: Change in social distancing-related stress in Denmark vs. Singapore.

Explanations of t-test types

Equal or unequal variance: Whether the two groups have (roughly) equal variance.

Equal variance: Groups “A” and “B” are caucasian 20-year old students from the same university; group “A” studies engineering and group “B” studies computer science.

Unequal variance: Group “A” is caucasian 20-year old students at the same engineering university. Group “B” is a random sample of 20-year olds from across the country.

Explanations of t-test types

Equal of unequal sample sizes: Whether the groups have the same number of values (applies only to unpaired groups).

Equal: Both groups have $N=30$.

Unequal: Group "A" is 13 Parkinson's patients; group "B" is 20 matched controls.

Formula for unequal N, unequal variances

$$t_{df} = \frac{\overline{x_1} - \overline{x_2}}{\sqrt{\frac{(n_1-1)s_1^2 + (n_2-1)s_2^2}{n_1 + n_2 - 2}} \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$$

$$df = n_1 + n_2 - 2$$

```

import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## generate the data
# parameters
n1 = 30 # samples in dataset 1
n2 = 40 # ...and 2
mu1 = 1 # population mean in dataset 1
mu2 = 1.2 # population mean in dataset 2

# generate the data
data1 = mu1 + np.random.randn(n1)
data2 = mu2 + np.random.randn(n2)

# show their histograms
plt.hist(data1,bins='fd',color=[1,0,0,.5],label='Data 1')
plt.hist(data2,bins='fd',color=[0,0,1,.5],label='Data 2')
plt.xlabel('Data value')
plt.ylabel('Count')
plt.legend()
plt.show()

## now for the t-test
t,p = stats.ttest_ind(data1,data2,equal_var=True)
df = n1+n2-2
print('t(%g) = %g, p=%g'%(df,t,p))

## a 2D space of t values
# ranges for t-value parameters
meandiffs = np.linspace(-3,3,80)
pooledvar = np.linspace(.5,4,100)

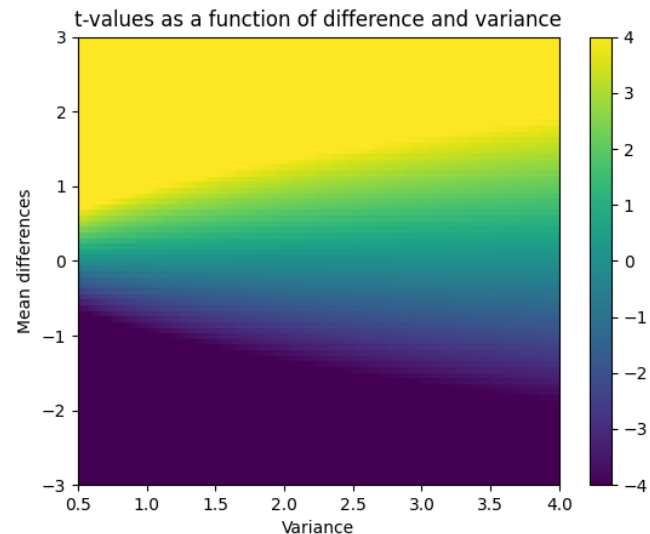
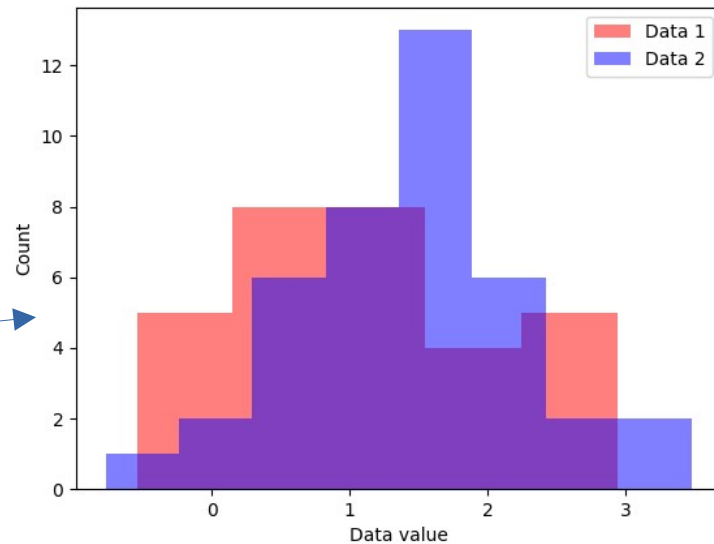
# group sample size
n1 = 40
n2 = 30

# initialize output matrix
allTvals = np.zeros((len(meandiffs),len(pooledvar)))

# loop over the parameters...
for meani in range(len(meandiffs)):
    for vari in range(len(pooledvar)):
        # t-value denominator
        df = n1 + n2 - 2
        s = np.sqrt(((n1-1)*pooledvar[vari] + (n2-1)*pooledvar[vari]) / df)
        t_den = s * np.sqrt(1/n1 + 1/n2)
        # t-value in the matrix
        allTvals[meani,vari] = meandiffs[meani] / t_den

plt.imshow(allTvals,vmin=-4,vmax=4,extent=[pooledvar[0],pooledvar[-1],meandiffs[0],meandiffs[-1]],origin='lower',aspect='auto')
plt.xlabel('Variance')
plt.ylabel('Mean differences')
plt.colorbar()
plt.title('t-values as a function of difference and variance')
plt.show()

```



Wilcoxon signed-rank (nonparametric t-test)

In this video, you will learn

- The purpose of the Wilcoxon sign-rank test.
- When to use the sign-rank test (one- or paired-sample).
- Differences to the Mann-Whitney U test.
- The formula of the Wilcoxon signed-rank test.

One-sample t-test: assumptions

1. Data are numeric (not categorical), ideally interval or ratio (discrete is probably OK).
2. Data are independent from each other.
3. Data are randomly drawn from the population to which generalization should be made.
4. Mean and standard deviation are valid descriptors of central tendency and dispersion (i.e., data are approximately normally distributed).

Wilcoxon signed-rank test

Nonparametric alternative to the one- or two-samples t-test.

Mainly used when the data do not conform to the normality assumption.

Tests for differences in medians instead of differences in means (median is insensitive to outliers).

Different nonparametric median t-tests

Test name	When to use
-----------	-------------

Wilcoxon signed-rank test Signed-rank test	One-sample Two dependent (paired) samples
---	--

Mann-Whitney U test Mann-Whitney-Wilcoxon U test Wilcoxon rank-sum test	Two independent samples
---	-------------------------

Algorithm to compute the Wilcoxon test

Step 1: Remove equal pairs

Remove equal pairs or data points that equal the H_0 value.

Step 2: Rank-transform diffs

Step 3: Sum ranks where $x > y$

Reasoning: Equal pairs do not contribute to the test either way.

Step 4: Convert to z

Algorithm to compute the Wilcoxon test

Step 1: Remove equal pairs

Step 2: Rank-transform diffs

Step 3: Sum ranks where $x > y$

Step 4: Convert to z

$$r = \text{rank}(|x - y|)$$

$x - y$ are the paired differences.

For a one-sample test, y is the H_0 value.

Algorithm to compute the Wilcoxon test

Step 1: Remove equal pairs

Step 2: Rank-transform diffs

Step 3: Sum ranks where $x > y$

Step 4: Convert to z

$$W = \sum (r \times (x > y))$$

Algorithm to compute the Wilcoxon test

Step 1: Remove equal pairs

Step 2: Rank-transform diffs

Step 3: Sum ranks where $x > y$

Step 4: Convert to z

$$Z = \frac{W - n(n+1)/4}{\sqrt{\frac{n(n+1)(2n+1)}{24}}}$$

n is the number of remaining pairs.

Z is normally distributed under H_0
and can be converted to a p-value.

```

import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats
## generate the data
N = 30
data1 = np.random.poisson(1.5,N)
data2 = np.random.poisson(1,N)
colors = 'kr'
for i in range(N):
    plt.plot([data1[i], data2[i]],[i, i],colors[int(data1[i]<data2[i])])
plt.plot(data1,np.arange(N),'ks',markerfacecolor='k',label='data1')
plt.plot(data2,np.arange(N),'ro',markerfacecolor='r',label='data2')
plt.ylabel('Data index')
plt.xlabel('Data value')
plt.legend()
plt.show()

```

```

## now for the test
t,p = stats.wilcoxon(data1,data2)
print('Wilcoxon z=%g, p=%g'%(t,p))

```

```

## now for the 2D space

```

```

# parameter ranges

```

```

Ns = np.arange(5,51)

```

```

lambdas = np.linspace(1,3,40)

```

```

# initialize output matrix

```

```

pvals = np.zeros((len(Ns),len(lambdas)))

```

```

for ni in range(len(Ns)):

```

```

    for li in range(len(lambdas)):

```

```

        # generate some data

```

```

        data1 = np.random.poisson(lambdas[0], Ns[ni])

```

```

        data2 = np.random.poisson(lambdas[li],Ns[ni])

```

```

        # compute the statistic

```

```

        t,p = stats.wilcoxon(data1,data2)

```

```

        # store the results

```

```

        pvals[ni,li] = -np.log(p)

```

```

# optional p-value thresholding

```

```

pvalthresh = .01

```

```

pvals[ pvals<-np.log(pvalthresh) ] = np.NaN

```

```

# now show in a heatmap!

```

```

plt.imshow(pvals,origin='lower',extent=[lambdas[0],lambdas[-1],Ns[0],Ns[-1]],aspect='auto')

```

```

plt.xlabel('lambda difference')

```

```

plt.ylabel('Sample size')

```

```

plt.title('Signed-rank test results: -log(p)')

```

```

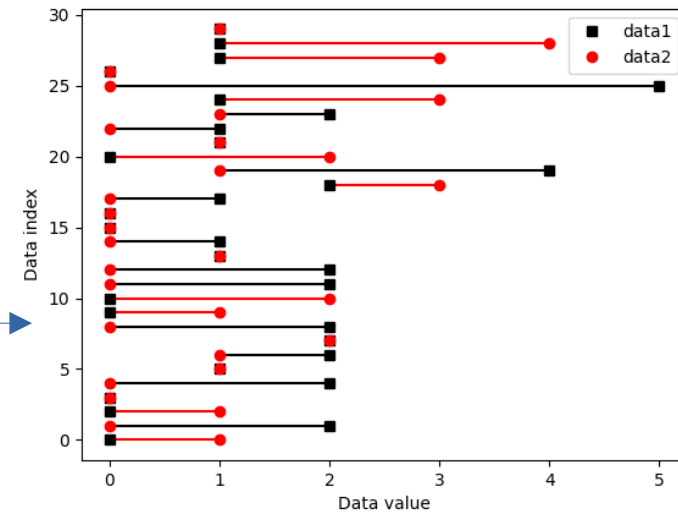
plt.colorbar()

```

```

plt.show()

```

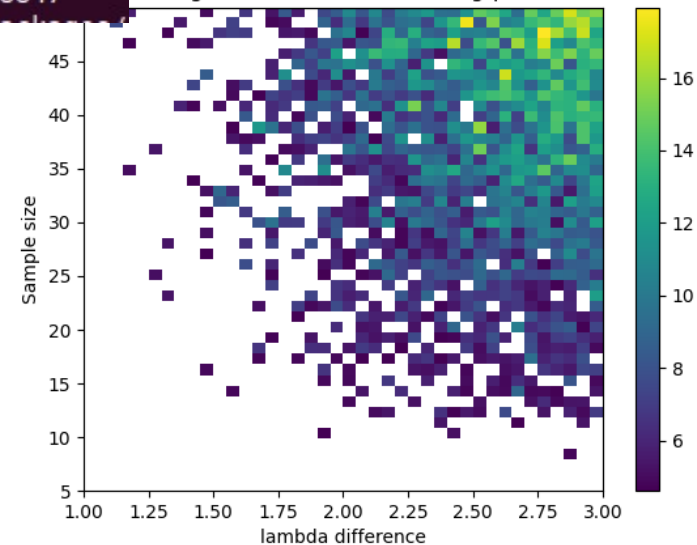


```

warnings.warn("Exact p-value
Wilcoxon z=95.5, p=0.478847
")

```

Signed-rank test results: $-\log(p)$



Mann-Whitney U test (nonparametric t-test)

In this video, you will learn

- The purpose of the Mann-Whitney U test.
- Reminder of difference with the Wilcoxon signed-rank test.
- The formula of the Mann-Whitney U test.

Mann-Whitney U test

Nonparametric alternative to the independent two-samples t-test.

Mainly used when the data do not conform to the normality assumption.

Tests for differences in medians instead of differences in means (median is insensitive to outliers). The two groups do not need to have the same sample size.

Different nonparametric median t-tests

Test name

When to use

Wilcoxon signed-rank test
Signed-rank test

One-sample
Two dependent (paired) samples

Mann-Whitney U test
Mann-Whitney-Wilcoxon U test
Wilcoxon rank-sum test

Two independent samples

Algorithm to compute the Mann-Whitney U

Step 1: Note the N's

Step 2: Pool data and rank

Step 3: Compute U

Step 4: Convert to z

x_f = Dataset with fewer points.

x_m = Dataset with more points.

n_f = Smaller sample size.

n_m = Larger sample size.

Algorithm to compute the Mann-Whitney U

Step 1: Note the N's

Step 2: Pool data and rank

$$r = \text{rank}(\{x_f, x_m\})$$

Step 3: Compute U

Step 4: Convert to z

$\{\dots\}$ is the set created by concatenating the group with more data points after the group with fewer data points.

Algorithm to compute the Mann-Whitney U

Step 1: Note the N's

Step 2: Pool data and rank

Step 3: Compute U

Step 4: Convert to z

$$U = \sum_{i=1}^{n_f} r_i$$

n_f is the smaller of the sample sizes.

Algorithm to compute the Mann-Whitney U

Step 1: Note the N's

Step 2: Pool data and rank

Step 3: Compute U

Step 4: Convert to z

$$Z = \dots$$

The formula is rather long...

Z is normally distributed under H_0
and can be converted to a p-value.


```

import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats

## generate the data
# the data (note the different sample sizes)
N1 = 30
N2 = 35

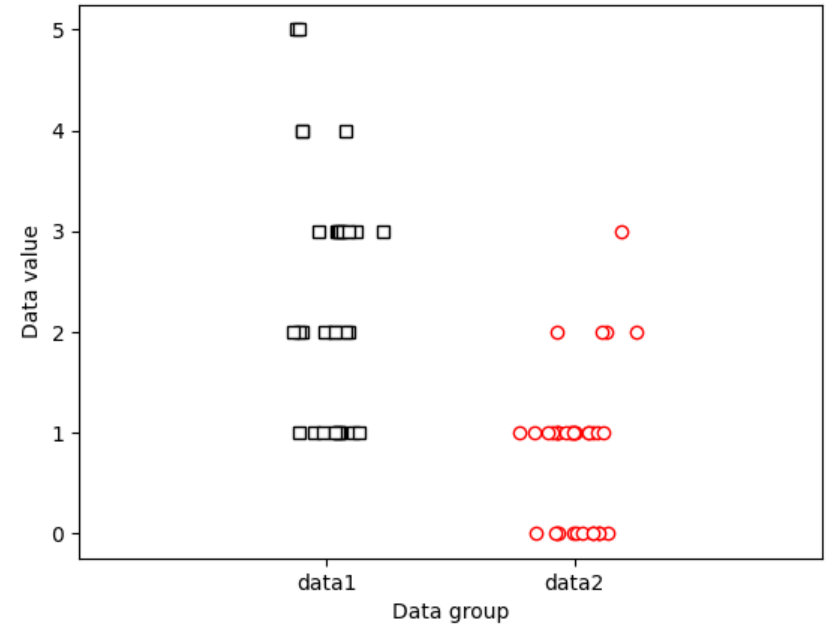
data1 = np.random.poisson(2,N1)
data2 = np.random.poisson(1,N2)

plt.plot(1+np.random.randn(N1)/10,data1,'ks',markerfacecolor='w')
plt.plot(2+np.random.randn(N2)/10,data2,'ro',markerfacecolor='w')

plt.xlim([0,3])
plt.xticks([1,2],labels=('data1','data2'))
plt.xlabel('Data group')
plt.ylabel('Data value')
plt.show()

## now for the test
U,p = stats.mannwhitneyu(data1,data2)
print(U,p)

```



```

warnings.warn("Unable to L
896.0 3.362043216594721e-07

```

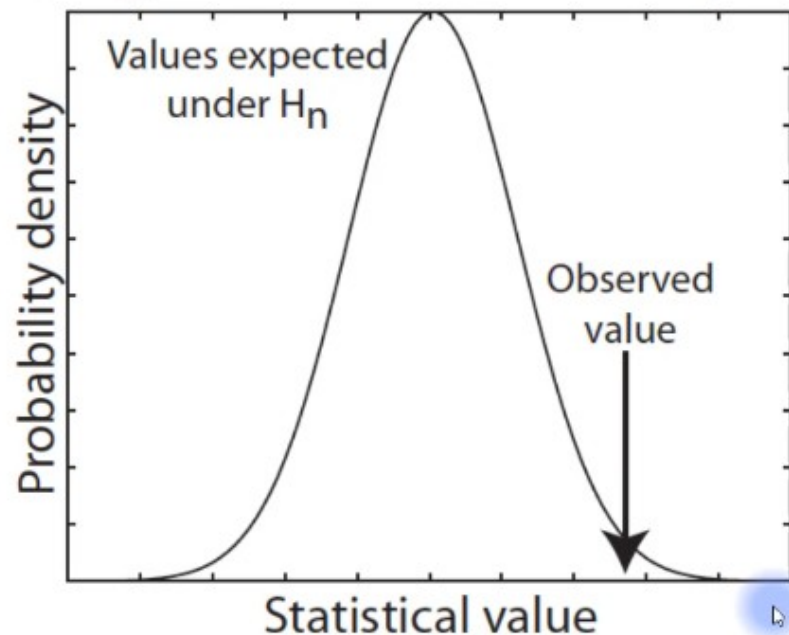
Permutation testing for t-test

In this video, you will learn

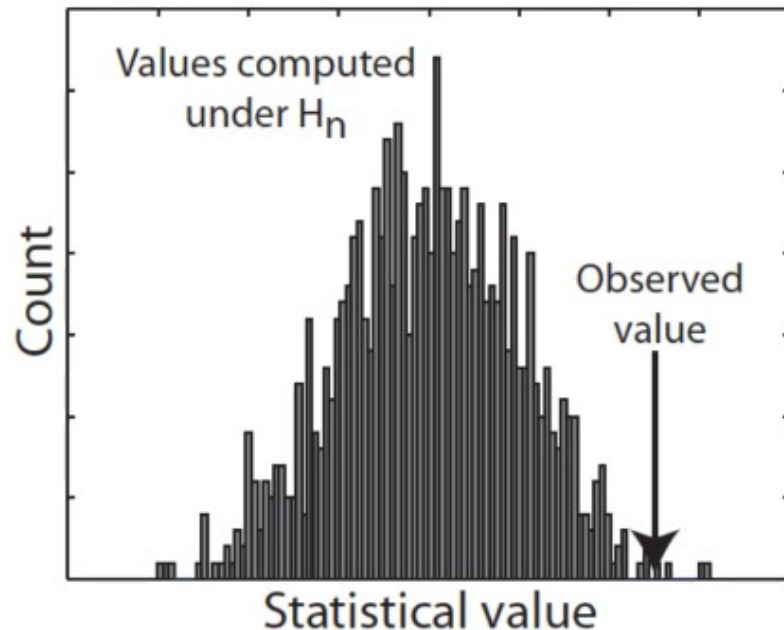
- The key difference between the parametric and permutation-based t-test significance (hint: H_0 !).
- How permutation testing works for t-tests.
- Two methods to compute a p-value.

Inference with permutation vs. parametric stats

A) Significance based on assumption



B) Significance based on data



Methodology:

- Calculate original statistic (e.g., $(\text{Mean_A} - \text{Mean_B})$).
- Shuffle group labels and recalculate the statistic thousands of times.
- Compare original statistic to the distribution of shuffled statistics to determine significance.

Use Cases: Permutation tests are highly effective for small sample sizes or when data violates t-test assumptions

Computational Cost: Permutation tests are computationally intensive compared to the instant output of t-tests, though this is rarely a limitation with modern computing.

Mechanism of permutation testing

Measured data



Pooled data



Random



Shuffled data



Mechanism of permutation testing

Shuffled data



Statistic value

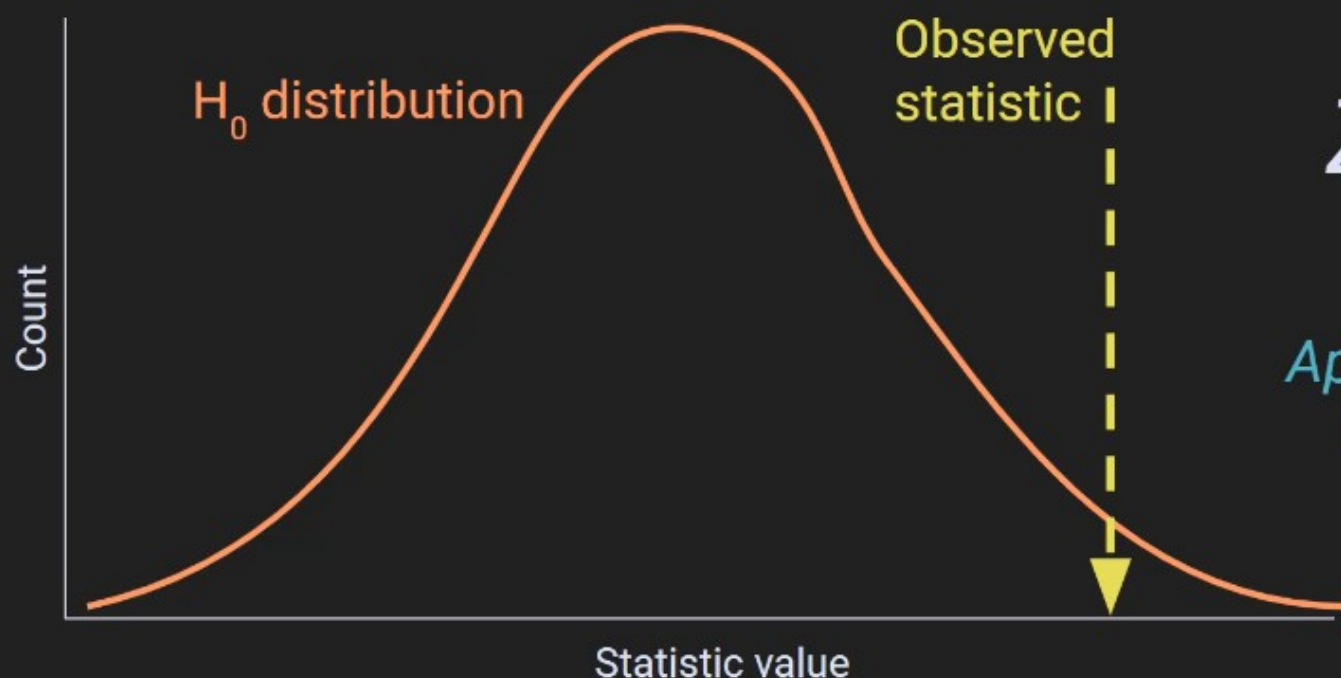
$$\frac{\Sigma - \Sigma}{\sigma \sigma}$$



Distribution



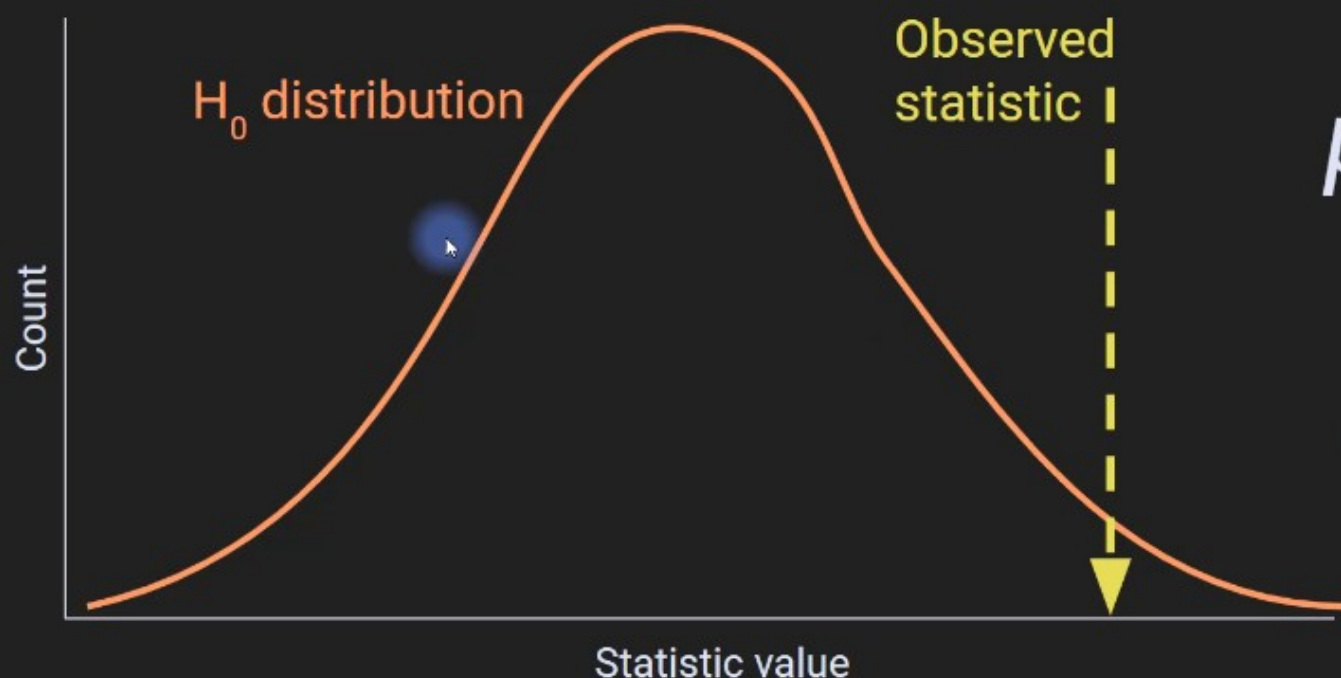
Evaluating significance (p-value)



$$Z = \frac{\text{obs} - E[H_0]}{\text{std}[H_0]}$$

*Appropriate for approximately
Gaussian H_0 distribution*

Evaluating significance (p-value)



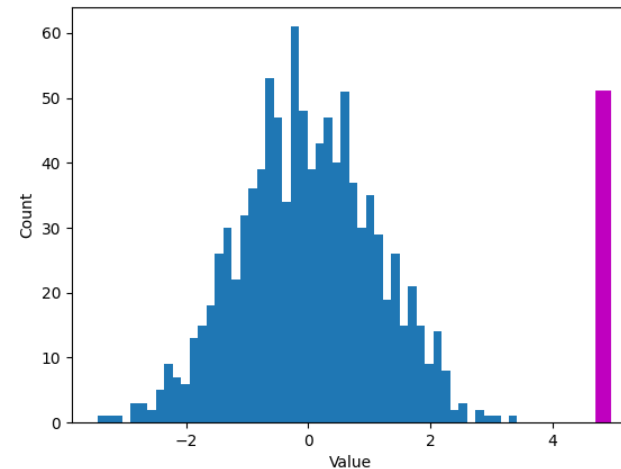
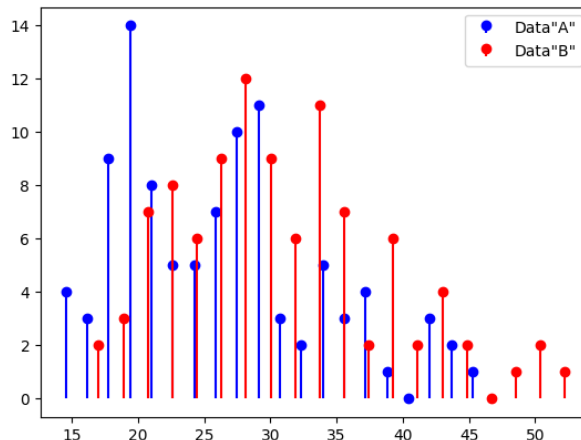
$$p_c = \frac{\sum(H_0 > \text{obs})}{N_{H_0}}$$

*Generally appropriate
(be careful of which tail)*

```

import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats
## simulate two distributions
# number of trials
N = 100
# dataset "A"
r = np.random.randn(N)
r[r>0] = np.log(1+r[r>0])
dataA = 26-r*10
# get histogram values for later comparison
yA,xA = np.histogram(dataA,20)
xA = (xA[:-1]+xA[1:])/2
# dataset "B"
r = np.random.randn(N)
r[r>0] = np.log(1+r[r>0])
dataB = 30-r*10
# get histogram values for later comparison
yB,xB = np.histogram(dataB,20)
xB = (xB[:-1]+xB[1:])/2
plt.stem(xA,yA,'b',markerfmt='bo',basefmt='-',label='Data"A"')
plt.stem(xB,yB,'r',markerfmt='ro',basefmt='-',label='Data"B"')
plt.legend()
plt.show()
## mix trials together
# concatenate trials
alldata = np.hstack((dataA,dataB))
# condition labels
conds = np.hstack((np.ones(N),2*np.ones(N)))
## generate one null hypothesis scenario
# random permutation
fakeconds = np.random.permutation(N*2)
# shuffled condition labels
fakeconds[fakeconds<N] = 1
fakeconds[fakeconds>N] = 2
# these two means should be different.
print([np.mean(alldata[conds==1]), np.mean(alldata[conds==2])])
# should these two be different?
print([np.mean(alldata[fakeconds==1]), np.mean(alldata[fakeconds==2])])
## and now a distribution of null hypothesis values
nPerms = 1000
permdiffs = np.zeros(nPerms)
for permi in range(nPerms):
    fconds = np.random.permutation(N*2)
    fconds[fconds<N] = 1
    fconds[fconds>N] = 2
    permdiffs[permi] = np.mean(alldata[fconds==2]) - np.mean(alldata[fconds==1])
# plot the distribution of H0 values
plt.hist(permdiffs,50)
# and plot the observed value on top
obsval = np.mean(alldata[conds==2]) - np.mean(alldata[conds==1])
plt.plot([obsval, obsval],[0, 50], 'm', linewidth=10)
plt.xlabel('Value')
plt.ylabel('Count')
plt.show()
## two methods of evaluating statistical significance
# Z-value
zVal = ( obsval-np.mean(permdiffs) ) / np.std(permdiffs,ddof=1)
p = 1-stats.norm.cdf(abs(zVal))
# p-value count
pCount = sum(permdiffs>obsval)/nPerms
print(p,pCount)

```



```

[25.899523221839093, 30.745378644423262]
[28.393190975785178, 28.251710890477188]
5.99748590157656e-06 0.0

```